

Cahier des Charges



MediaStock

2025

Table des matières

1. Introduction et Présentation du Projet.....	3
1.1 Présentation Générale.....	3
1.2 Finalité du Document.....	3
2. Contexte et Problématique.....	3
2.1 Situation Actuelle.....	3
2.2 Problèmes Identifiés.....	3
2.3 Enjeux et Opportunités.....	4
3. Objectifs du Projet.....	4
3.1 Objectif Principal.....	4
3.2 Objectifs Spécifiques.....	4
3.3 Bénéfices Attendus.....	4
4. Analyse des Besoins Fonctionnels.....	4
4.1 Hiérarchisation des Fonctionnalités.....	5
5. Analyse des Besoins Techniques.....	7
5.1 Infrastructure et Environnement de Développement.....	7
6. Architecture et Conception Technique.....	9
6.1 Architecture globale.....	9
6.2 Structure de la base de données.....	9
6.3 Gestion des QR codes.....	9
6.4 Sécurité.....	10
6.5 Maquettage et conception d'interface avec Figma.....	10
7. Méthodologie de Développement.....	10
7.1 Approche itérative et incrémentale.....	10
7.2 Organisation des itérations.....	11
7.3 Gestion de version.....	11
8. Plan de Tests et Validation.....	12
8.1 Méthodologie des tests.....	12
8.2 Tests fonctionnels détaillés.....	12
8.3 Procédure de recette finale.....	13
9. Déploiement et Mise en Production.....	13
9.1 Préparation de l'Environnement.....	13
9.2 Configuration pour la Production.....	14
9.3 Procédure de Déploiement.....	14
9.4 Stratégie de Sauvegarde.....	15
9.5 Surveillance et Maintenance.....	15
10. Planning Prévisionnel.....	15
11. Livrables Attendus.....	16
11.1 Documentation.....	16
11.2 Code Source et Scripts.....	16
11.3 Documentation Technique.....	16
11.4 Documentation Utilisateur.....	16
11.5 Éléments Opérationnels.....	17
11.6 Application Déployée.....	17
Annexes.....	18

1. Introduction et Présentation du Projet

1.1 Présentation Générale

Ce document constitue le cahier des charges détaillé de l'application de gestion de prêts de matériel pour l'établissement scolaire. L'objectif de ce document est de définir de manière exhaustive et précise l'ensemble des spécifications, des fonctionnalités, des contraintes techniques et des modalités de réalisation du projet. Ce cahier des charges servira de référence tout au long du développement de l'application et permettra de valider chaque étape de réalisation auprès du client et de l'équipe pédagogique.

1.2 Finalité du Document

Ce cahier des charges a pour vocation de décrire dans le détail comment l'application sera conçue, développée, testée et déployée. Chaque section explique non seulement ce qui doit être fait, mais également comment cela sera réalisé et comment nous nous assurerons que le résultat correspond aux attentes du client. Ce document doit être accessible et compréhensible par tous les acteurs du projet, qu'ils soient techniques ou non.

2. Contexte et Problématique

2.1 Situation Actuelle

Actuellement, l'école gère les prêts de matériel informatique et multimédia à l'aide de fiches papier. Cette méthode présente plusieurs inconvénients majeurs qui impactent le quotidien de l'équipe administrative et des utilisateurs. Les fiches papier peuvent être perdues, mal remplies ou oubliées, ce qui entraîne une perte de traçabilité importante. Il devient alors difficile de savoir qui a emprunté quel matériel, à quelle date, et si celui-ci a été restitué ou non.

Le matériel concerné comprend une grande variété d'équipements tels que des claviers, des souris, des adaptateurs divers, des microphones, des cartes SD, et d'autres périphériques informatiques. Ces équipements sont régulièrement empruntés par les étudiants et les intervenants pour leurs projets, leurs travaux pratiques ou leurs présentations.

2.2 Problèmes Identifiés

Les problèmes principaux rencontrés avec le système actuel sont multiples. Premièrement, la traçabilité est insuffisante car il est impossible de consulter rapidement l'historique des prêts ou de savoir en temps réel quel matériel est disponible. Deuxièmement, les opérations de prêt et de restitution prennent du temps car elles nécessitent de remplir manuellement des formulaires papier. Troisièmement, il n'existe aucun système d'alerte pour les retards de restitution, ce qui peut entraîner des pertes ou des blocages pour d'autres utilisateurs en attente. Enfin, la gestion de l'inventaire est complexe et ne permet pas d'avoir une vision globale du stock disponible.

2.3 Enjeux et Opportunités

La digitalisation de la gestion des prêts représente une opportunité importante pour l'école. Elle permettra d'améliorer l'efficacité opérationnelle de l'équipe administrative, d'offrir une meilleure transparence aux utilisateurs qui pourront consulter la disponibilité du matériel à tout moment, et de garantir une traçabilité complète pour prévenir les pertes et gérer les retards. De plus, cette solution moderne s'inscrit dans la démarche de transformation numérique de l'établissement.

3. Objectifs du Projet

3.1 Objectif Principal

L'objectif principal de ce projet est de concevoir et développer une application web mobile-first pour gérer l'ensemble du cycle de vie des prêts de matériel. L'application devra :

- Remplacer complètement le système papier actuel.
- Offrir une expérience utilisateur fluide et rapide.
- Permettre à l'équipe administrative de gérer les prêts, les restitutions et l'inventaire.
- Donner une visibilité sur la disponibilité du matériel sans nécessiter de connexion.

3.2 Objectifs Spécifiques

Plusieurs objectifs spécifiques structurent ce projet.

- Réduire le temps nécessaire pour effectuer une opération de prêt.
- Garantir une traçabilité complète de tous les mouvements de matériel avec horodatage précis.
- Optimiser l'interface pour une utilisation sur smartphone pour permettre les opérations directement sur le terrain.
- Intégrer un système de scan de QR codes pour identifier rapidement le matériel.

3.3 Bénéfices Attendus

- Gain de temps pour l'équipe administrative sur les tâches répétitives.
- Réduction des erreurs de saisie et des pertes de matériel.
- Amélioration de la gestion patrimoniale et professionnalisation des processus internes.

4. Analyse des Besoins Fonctionnels

L'analyse des besoins fonctionnels constitue le cœur de ce cahier des charges. Son objectif principal est de définir précisément ce que l'application MédiaStock doit permettre de faire, en décrivant chaque fonctionnalité de manière exhaustive et compréhensible. Cette section servira de référence tout au long du développement pour garantir que chaque composant de l'application réponde aux attentes de l'école et facilite réellement le travail quotidien de l'équipe média/administration.

4.1 Hiérarchisation des Fonctionnalités

Les fonctionnalités de MédiaStock sont organisées selon trois niveaux de priorité qui guideront l'ordre de développement et l'allocation des ressources:

- **MUST** (indispensables) : Socle minimum pour que l'application soit opérationnelle.
- **SHOULD** (importantes) : Améliorent significativement l'expérience utilisateur.
- **COULD** (bonus) : Apportent une valeur ajoutée mais ne sont pas critiques pour le lancement.

4.2.1 Fonctionnalités **MUST**

1. Gestion complète de l'inventaire (CRUD) :

- Créer, consulter, modifier et supprimer des fiches pour chaque matériel.
- Générer automatiquement un code QR unique pour chaque matériel.
- Vérifier l'unicité des items pour éviter les doublons.

2. Création et enregistrement des prêts :

La création de prêts représente l'action la plus fréquente dans l'utilisation quotidienne de MédiaStock :

- Enregistrer un prêt en scannant le QR code du matériel.
- Saisir les informations de l'emprunteur (nom, prénom, rôle).
- Définir une date de retour prévue.
- Bloquer les prêts si le matériel est déjà emprunté.

3. Enregistrement des restitutions :

La restitution de matériel est l'opération qui clôture le cycle de prêt et libère le matériel pour un nouvel emprunt.

- Scanner le QR code du matériel retourné.
- Enregistrer la date et l'heure exactes de la restitution.
- Ajouter un commentaire sur l'état du matériel.
- Mettre à jour le statut de disponibilité du matériel.

4. Système d'authentification unique :

L'authentification est le mécanisme qui protège l'accès aux fonctions administratives de MédiaStock tout en restant simple d'utilisation pour l'équipe. Contrairement à de nombreux systèmes d'entreprise, MédiaStock n'utilisera pas de système SSO (Single Sign-On) connecté à l'infrastructure informatique de l'école. Cette décision stratégique vise à maintenir l'application totalement indépendante et simple, évitant les complexités techniques et les dépendances qui pourraient ralentir le développement ou créer des problèmes de maintenance.

- Un compte administrateur partagé pour toute l'équipe.
- Stockage sécurisé des mots de passe (hashés avec bcrypt).
- Protection contre les attaques par force brute.

4.2.2 Fonctionnalités *SHOULD*

1. Gestion des dates d'échéance et signalement des retards :

La gestion des dates de retour prévues et le signalement automatique des retards constituent une amélioration majeure par rapport au système papier actuellement utilisé

- Définir une date de retour prévue pour chaque prêt.
- Identifier automatiquement les prêts en retard.
- Afficher visuellement les retards dans l'interface.

2. Génération et impression des QR codes :

La génération automatique et l'impression facilitée des QR codes sont essentielles pour le déploiement initial de l'application et pour l'ajout futur de nouveau matériel.

- Générer automatiquement un QR code pour chaque nouvel item.
- Permettre l'impression des QR codes sous forme d'étiquettes PDF.

4.2.3 Fonctionnalités *COULD*

Ces fonctionnalités représentent des améliorations intéressantes qui pourront être développées après la mise en production initiale de l'application, selon les retours d'utilisation et les besoins identifiés. Elles ne sont pas nécessaires au lancement mais enrichiraient l'expérience utilisateur et les capacités de gestion.

1. Contrôle d'inventaire par scan :

Un mode "Inventaire" permettrait de scanner successivement tous les objets physiquement présents et de comparer cette liste avec l'inventaire théorique de la base de données, identifiant ainsi les écarts (objets manquants ou surplus). Cette fonctionnalité faciliterait les opérations de contrôle périodique du parc matériel.

2. Tableaux de bord et statistiques :

Un module statistique pourrait afficher des informations utiles comme le taux d'utilisation de chaque catégorie de matériel, les objets les plus empruntés, la durée moyenne des prêts, le taux de retard par type d'emprunteur, etc. Ces données aideraient à optimiser la composition du parc matériel en identifiant les besoins réels.

3. Gestion multi-comptes et journal d'audit :

L'évolution vers des comptes individuels pour chaque membre de l'équipe administrative, accompagnée d'un journal traçant qui a effectué quelle action et quand, renforcerait la traçabilité et la responsabilisation. Cette fonctionnalité deviendrait pertinente si l'équipe s'agrandit ou si des besoins de contrôle plus fins émergent.

5. Analyse des Besoins Techniques

L'objectif de cette section est de définir précisément l'environnement technique dans lequel l'application MédiaStock sera développée et déployée. Cette analyse permet d'établir les fondations technologiques du projet, en identifiant les outils, les frameworks, les langages de programmation et les infrastructures nécessaires pour répondre aux exigences fonctionnelles définies précédemment.

5.1 Infrastructure et Environnement de Développement

5.1.1 Conteneurisation avec Docker

L'application MédiaStock sera développée et déployée à l'aide de Docker, utilisé comme plateforme de conteneurisation.

- **Cohérence des environnements :**

Utilisation de Docker afin de garantir la cohérence entre les environnements de développement, test et production.

- **Architecture des conteneurs :**

Mise en place de trois conteneurs principaux :

- un conteneur serveur web Apache/PHP,
- un conteneur base de données MySQL,
- et un conteneur phpMyAdmin (utilisé uniquement pour le développement).

Le choix de Docker s'appuie sur le fait qu'il facilite la collaboration entre les développeurs, permet un partage simplifié des configurations, et réduit considérablement le temps d'installation d'un nouvel environnement de travail.

5.1.2 Base de données : MySQL

La base de données relationnelle MySQL a été retenue pour la gestion de l'ensemble des données de l'application MédiaStock.

Ce choix repose sur plusieurs critères techniques et contextuels.

MySQL est un système de gestion de base de données (SGBD) éprouvé, particulièrement adapté aux applications web de petite à moyenne envergure, comme celle du présent projet.

Avec un volume initial estimé entre 50 et 100 items de matériel, MySQL offre les performances nécessaires, tout en garantissant une simplicité d'administration et une fiabilité reconnue.

5.1.3 Développement back-end : PHP

Le langage PHP sera utilisé pour le développement de la logique métier côté serveur. La version retenue permettra de bénéficier d'améliorations significatives en termes de performances, de typage strict et de fonctionnalités syntaxiques modernes, facilitant ainsi la maintenance et l'évolutivité du code.

PHP a été retenu car il s'intègre naturellement avec MySQL et offre nativement toutes les fonctionnalités nécessaires à la gestion des sessions utilisateurs, au traitement des requêtes HTTP et à la manipulation des données.

- **Architecture** : MVC léger implémenté en PHP procédural.
- **Sécurité et sessions** : Gestion de l'authentification et des sessions avec PHP natif.
- **QR Codes** : Génération de QR codes à l'aide d'une bibliothèque PHP (par exemple PHP QR Code).

5.1.4 Développement front-end

- L'interface utilisateur sera développée à l'aide des technologies web standard suivantes : HTML5, CSS3 et JavaScript (ES6+).
- L'application MédiaStock étant principalement utilisée sur smartphone par l'équipe média/administration lors des opérations de prêt et de restitution, une approche mobile-first sera privilégiée.
- Le framework CSS Bootstrap 5 sera utilisé afin de garantir un design responsive et une ergonomie adaptée aux différents formats d'écran.
- Le scan des QR codes sera effectué directement dans le navigateur grâce à des bibliothèques JavaScript modernes telles que html5-qrcode ou jsQR.

5.2 Sécurité et conformité

- **Communication sécurisée** :

L'ensemble de l'application sera servi exclusivement via le protocole HTTPS, garantissant le chiffrement de toutes les communications entre le navigateur et le serveur.

Un certificat SSL/TLS sera mis en place afin d'assurer la confidentialité et l'intégrité des échanges.

- **Protection contre les vulnérabilités** :

L'application sera développée en appliquant rigoureusement les bonnes pratiques de sécurité afin de prévenir les vulnérabilités les plus courantes :

- Utilisation de requêtes SQL préparées (PDO) pour éviter les injections SQL.
- Échappement systématique des données affichées (*htmlspecialchars*) pour prévenir les attaques XSS.

- **Conformité RGPD** :

L'application respectera le Règlement Général sur la Protection des Données (RGPD), notamment :

- Collecte minimale des données personnelles, limitée aux besoins strictement nécessaires.
- Politique de conservation des données avec suppression automatique après 24 mois.

6. Architecture et Conception Technique

L'objectif de cette section est de définir précisément l'architecture technique de l'application MédiaStock afin de garantir sa robustesse, sa maintenabilité et sa scalabilité.

6.1 Architecture globale

L'application MédiaStock repose sur une architecture trois-tiers classique, qui sépare clairement la présentation, la logique métier et la persistance des données.

Cette approche favorise une meilleure organisation du code, facilite la maintenance et permet de faire évoluer indépendamment chaque composant de l'application.

- **Présentation** : Interfaces web responsives développées en HTML, CSS et JavaScript.
- **Application** : Logique métier implémentée en PHP, selon une architecture MVC léger.
- **Données** : Gestion et stockage assurés par la base de données MySQL

6.2 Structure de la base de données

Le modèle relationnel de la base de données est structuré autour de sept tables principales.

Une indexation a été mise en place afin d'optimiser les performances des requêtes et la rapidité d'accès aux données.

Tables principales :

- **ITEM** : informations sur chaque matériel (nom, catégorie, QR code, statut).
- **PRÊT** : historique des prêts (item, emprunteur, dates de prêt et de restitution).
- **EMPRUNTEUR** : données relatives aux emprunteurs (nom, prénom, type).
- **ADMIN** : comptes administrateurs (identifiant, mot de passe haché).
- **FORMATION** : représente les différentes **formations de l'école**.
- **CATEGORIE** et **SOUS_CATEGORIE** : définissent les **catégories et sous-catégories** associées à chaque item.

Le MCD de la base de données est présenté à la [Figure 1](#) et le MLD de la base de données est présenté à la [Figure 2](#) de ce cahier des charges.

6.3 Gestion des QR codes

La gestion des QR codes constitue un élément central de l'architecture de l'application, car elle conditionne directement la rapidité et la fiabilité des opérations de prêt et de restitution du matériel.

Le système repose sur deux processus complémentaires :

- **Génération** : Utilisation d'une bibliothèque PHP (par exemple *PHP QR Code*) pour créer des QR codes uniques associés à chaque item de matériel.
- **Scan** : Intégration d'une bibliothèque JavaScript (*html5-qrcode*) permettant le scan des QR codes directement depuis la caméra du smartphone.

6.4 Sécurité

La sécurité de l'application MédiaStock repose sur plusieurs couches de protection appliquées à différents niveaux de l'architecture.

Bien que l'application ne traite pas de données hautement sensibles, elle doit néanmoins respecter les bonnes pratiques de sécurité web afin de protéger les informations personnelles des emprunteurs et de garantir l'intégrité du système d'inventaire.

- **Authentification :**
Gestion des sessions utilisateurs via PHP, avec stockage sécurisé des identifiants et mots de passe hachés.
- **Protection des données :**
Chiffrement des communications entre le client et le serveur via le protocole HTTPS, et mise en place de sauvegardes automatiques pour prévenir toute perte de données.

6.5 Maquettage et conception d'interface avec Figma

Avant le début du développement, l'ensemble des interfaces de l'application sera maqueté à l'aide de Figma.

Cette étape de conception visuelle poursuit plusieurs objectifs essentiels :

- Valider l'ergonomie et le parcours utilisateur avant tout investissement en développement.
- Faciliter la communication avec les parties prenantes en offrant une vision claire de l'apparence finale de l'application.
- Fournir aux développeurs des spécifications visuelles précises, incluant les dimensions, espacements, couleurs et typographies.

Les prototypes interactifs réalisés sous Figma permettront de simuler la navigation entre les écrans, offrant ainsi une prévisualisation réaliste de l'expérience utilisateur finale.

La maquette de l'application est présentée à la [Figure 3](#) de ce cahier des charges.

7. Méthodologie de Développement

L'objectif de cette section est de définir précisément la démarche méthodologique qui sera adoptée tout au long du développement de l'application MédiaStock. Cette méthodologie doit garantir une progression structurée et maîtrisée du projet, tout en respectant les contraintes calendaires imposées par l'école, notamment la livraison avant le 15 décembre 2025. La méthodologie choisie permettra d'organiser le travail hebdomadaire des vendredis, d'assurer une validation continue des fonctionnalités développées, et de maintenir une communication fluide avec les parties prenantes. Cette approche méthodique constitue la colonne vertébrale du projet et servira de référence pour toutes les décisions techniques et organisationnelles prises pendant les phases de développement.

7.1 Approche itérative et incrémentale

Le développement de l'application MédiaStock suivra une approche itérative et incrémentale, inspirée des méthodologies agiles, et adaptée au contexte spécifique d'un projet étudiant avec des séances de travail hebdomadaires.

Cette approche consiste à découper le projet en cycles de développement successifs, appelés itérations (ou *sprints*), au cours desquels chaque cycle aboutit à une version fonctionnelle de l'application, enrichie de nouvelles fonctionnalités par rapport à la version précédente.

- **Planification du cycle** : 8 itérations hebdomadaires (chaque vendredi).
- **Contenu d'une itération** : planification, développement, tests et documentation.

7.2 Organisation des itérations

Chaque vendredi, le travail sera organisé selon un schéma récurrent structurant la journée en plusieurs phases complémentaires.

Ce fonctionnement garantit une utilisation optimale du temps disponible, tout en maintenant une vision claire de l'avancement du projet et des objectifs à atteindre.

Répartition prévisionnelle des itérations

- **Semaines 1–2** : Conception, création des maquettes Figma et définition du modèle de base de données.
- **Semaines 3–4** : Mise en place de l'infrastructure Docker et développement des pages de base (accueil, connexion, tableau de bord).
- **Semaines 5–6** : Développement des fonctionnalités cœur (gestion d'inventaire, prêts, restitutions).
- **Semaines 7–8** : Implémentation des fonctionnalités avancées (gestion des retards, statistiques), tests et documentation.

7.3 Gestion de version

La gestion de version du code source constitue un pilier essentiel de la méthodologie de développement adoptée.

L'ensemble du code de l'application MédiaStock sera hébergé dans un dépôt Git, garantissant la traçabilité des modifications, la collaboration entre développeurs et la sécurisation du code.

- **Structure des branches** :
 - `main` : branche de référence contenant le code stable.
 - `nom_du_developpeur` : branche dédiée au travail individuel d'un développeur.
 - `feature/*` : branches dédiées au développement de nouvelles fonctionnalités.

- **Convention de messages de commit** :

Utilisation d'un format structuré inspiré des Conventional Commits, par exemple :

- `feat`: pour une nouvelle fonctionnalité,
- `fix`: pour une correction,
- `docs`: pour une mise à jour de documentation.

8. Plan de Tests et Validation

L'objectif de cette section est de définir précisément la stratégie de tests appliquée tout au long du développement de l'application MédiaStock, afin de garantir que chaque fonctionnalité développée réponde aux attentes du client et fonctionne de manière fiable et cohérente dans toutes les conditions d'utilisation.

8.1 Méthodologie des tests

La stratégie de tests de l'application MédiaStock repose sur une approche pyramidale, combinant plusieurs niveaux de tests complémentaires afin de couvrir l'ensemble des aspects fonctionnels et techniques de l'application.

- **Tests unitaires** : Vérification du bon fonctionnement des fonctions PHP isolées.
- **Tests d'intégration** : Validation des interactions entre les différents composants de l'application.
- **Tests fonctionnels** : Exécution de scénarios utilisateurs complets pour s'assurer de la conformité globale du système.

8.2 Tests fonctionnels détaillés

8.2.1 Authentification

Objectif du test :

Valider que le système d'authentification protège efficacement l'accès aux fonctions administratives, tout en offrant une expérience de connexion fluide pour les utilisateurs autorisés.

Scénarios testés :

- Connexion réussie et échouée.
- Gestion et persistance des sessions utilisateurs.

8.2.2 Gestion d'inventaire – Création d'un item

Objectif du test :

Valider que l'administrateur peut créer un nouvel item dans l'inventaire avec l'ensemble des informations requises, et que le QR code est généré automatiquement et correctement.

Scénarios testés :

- Création, modification et suppression d'items.
- Génération automatique et vérification des QR codes.

8.2.3 Prêts et restitutions

Objectif du test :

Valider le processus complet de gestion des prêts, depuis le scan du QR code jusqu'à l'enregistrement dans la base de données, en s'assurant que l'opération soit réalisable en moins de dix secondes et vérifier également que la restitution d'un matériel clôture correctement le prêt et rend l'item à nouveau disponible.

Scénarios testés :

- Scan et enregistrement des QR codes.
- Gestion des prêts et restitutions.
- Détection et suivi des retards.

8.2.4 Sécurité

Objectif du test :

Vérifier que l'application est protégée contre les principales vulnérabilités web.

Scénarios testés :

- Protection contre les injections SQL.
- Prévention des attaques XSS (Cross-Site Scripting).

8.3 Procédure de recette finale

La recette finale sera organisée avec les représentants de l'école au cours de la dixième semaine du planning de développement.

Une checklist exhaustive sera parcourue de manière méthodique afin de vérifier l'ensemble des fonctionnalités prioritaires classées selon les niveaux MUST et SHOULD.

Chaque test validé sera consigné, signé et approuvé par les parties prenantes.

- **Critères de validation :**

- Validation de 100 % des fonctionnalités MUST.
- Validation d'au moins 80 % des fonctionnalités SHOULD.

- **Participants :**

- Tests réalisés conjointement avec les représentants de l'école et l'équipe de développement.

9. Déploiement et Mise en Production

L'objectif de cette section est de définir la stratégie de déploiement et de mise en production de l'application MédiaStock. Le déploiement marque la transition entre l'environnement de développement local (Docker) et l'environnement de production hébergé sur la plateforme **Alwaysdata**.

9.1 Préparation de l'Environnement

La préparation de l'environnement de production consiste à configurer l'espace d'hébergement **Alwaysdata** pour accueillir durablement l'application.

Choix de l'hébergement

L'application sera hébergée sur **Alwaysdata**, une plateforme d'hébergement mutualisé qui fournit directement :

- un environnement Apache/PHP 8.x,
- une base de données MySQL 8.0,
- un accès HTTPS automatique,

- un panneau d'administration pour la gestion de la base de données.

Ce choix simplifie le déploiement en évitant l'administration d'un serveur complet, tout en restant compatible avec l'architecture PHP/MySQL du projet.

Configuration technique

Aucune installation de serveur n'est nécessaire. Alwaysdata fournit l'environnement d'exécution clé en main. La configuration se limite à :

- adapter le fichier *config/env.php* avec les paramètres de connexion fournis par Alwaysdata,
- vérifier que le DocumentRoot pointe vers le dossier public/,
- s'assurer que le fichier .htaccess est actif.

Sécurité

Le HTTPS est géré automatiquement par Alwaysdata (certificat SSL/TLS inclus).

- Les paramètres sensibles (identifiants de base de données) sont isolés dans le fichier *config/env.php*, non versionné dans Git.

9.2 Configuration pour la Production

La configuration de MédiaStock en production diffère de celle utilisée en développement :

- En **développement**, l'application tourne dans des conteneurs Docker (Apache, PHP, MySQL, phpMyAdmin).
- En **production sur Alwaysdata**, l'application est déployée directement sur l'infrastructure de l'hébergeur, sans Docker Compose.

Variables et sécurité

Les paramètres de connexion à la base de données sont définis dans le fichier *config/env.php*, non versionné. Ce fichier contient :

- l'hôte MySQL fourni par Alwaysdata,
- le nom de la base de données,
- l'identifiant et le mot de passe de connexion.

PhpMyAdmin n'est pas exposé en production : l'administration de la base se fait via l'interface Alwaysdata.

9.3 Procédure de Déploiement

Se connecter à l'interface d'administration Alwaysdata et créer la base de données MySQL.

1. Téléverser le code source sur l'espace d'hébergement via FTP/SFTP ou Git.
2. Adapter le fichier *config/env.php* avec les paramètres de connexion Alwaysdata.
3. Importer le script *sql/init.sql* via l'outil d'administration MySQL d'Alwaysdata.
4. Vérifier que le DocumentRoot pointe vers *public/* et que le *.htaccess* est actif.
5. Tester l'accès à l'application via l'URL publique et valider le bon fonctionnement du HTTPS.

9.4 Stratégie de Sauvegarde

La mise en place d'une stratégie de sauvegarde robuste constitue une exigence essentielle pour garantir la pérennité des données de l'application MédiaStock.

Une perte de données serait catastrophique, car elle entraînerait la disparition de l'historique des prêts et pourrait rendre impossible l'identification du matériel emprunté et de ses utilisateurs.

La stratégie de sauvegarde doit donc être conçue pour protéger l'application contre tout type de sinistre, qu'il s'agisse d'une panne matérielle, d'une corruption de données logicielle, d'une erreur humaine ou d'une attaque informatique (ex. ransomware).

- **Sauvegardes de la base de données :**
 - Export manuel ou automatique via l'interface Alwaysdata ou phpMyAdmin.
 - Sauvegardes quotidiennes recommandées, conservées pendant 3 mois.
- **Stockage des sauvegardes :**
 - Local sur le serveur principal.
 - Distant sur un serveur de secours.
 - Cloud (optionnel) pour une redondance supplémentaire.

9.5 Surveillance et Maintenance

Une fois l'application déployée en production, la surveillance continue de son bon fonctionnement devient indispensable pour détecter rapidement les problèmes potentiels et garantir une disponibilité maximale du service. La surveillance s'articulera autour de plusieurs mécanismes complémentaires qui collectent des métriques techniques et alertent l'administrateur en cas d'anomalie.

- **Monitoring :**
Suivi de la disponibilité via l'interface Alwaysdata (logs, métriques, alertes).
- **Maintenance :**
 - Mises à jour du code déployées via FTP/SFTP ou Git.
 - Renouvellement automatique du certificat SSL/TLS géré par Alwaysdata.
 - Vérification périodique du bon fonctionnement des fonctionnalités principales (connexion, prêt, restitution).

10. Planning Prévisionnel

L'objectif de cette section est de définir précisément le calendrier de réalisation du projet MédiaStock en tenant compte de la contrainte calendaire imposée par l'école : une livraison complète avant le 15 décembre 2025.

Ce planning prévisionnel constitue la feuille de route opérationnelle du projet et permet de répartir les différentes tâches de développement, de test et de documentation sur les huit vendredis de travail disponibles entre début octobre et mi-décembre.

Le planning prévisionnel est illustré par le diagramme de Gantt, présenté à la [Figure 4](#) de ce cahier des charges.

Découpage Temporel

Le projet MédiaStock s'étendra sur une période totale de dix semaines, du 3 octobre au 15 décembre 2025, avec huit vendredis de travail intensif.

Ce découpage est illustré par le diagramme de Kanban, présenté à la [Figure 5](#) et [Figure 6](#) de ce cahier des charges.

11. Livrables Attendus

L'objectif de cette section est de définir précisément l'ensemble des documents, fichiers et éléments livrables qui seront remis à l'école à l'issue du projet MédiaStock.

Ces livrables constituent la contrepartie formelle du travail réalisé et permettent de vérifier la conformité du projet aux exigences du cahier des charges.

11.1 Documentation

Le premier ensemble de livrables concerne la documentation de conception, retraçant l'ensemble des décisions prises pendant la phase initiale du projet.

Elle servira de référence technique pour comprendre l'architecture, les choix technologiques et la structure de l'application.

- Cahier des charges validé.
- Maquettes Figma (versions mobile, tablette et desktop).
- Schéma de base de données (Modèle Entité-Relation et dictionnaire de données).

11.2 Code Source et Scripts

Le deuxième ensemble de livrables concerne le code source complet de l'application ainsi que tous les scripts et fichiers de configuration nécessaires à son installation, son déploiement et son exploitation.

- Dépôt Git contenant l'ensemble du code PHP, HTML, CSS et JavaScript.
- Script SQL d'initialisation de la base de données.
- Fichiers de configuration Docker (docker-compose.yml, .env) pour la mise en place de l'environnement applicatif.

11.3 Documentation Technique

Le troisième ensemble de livrables concerne la documentation technique, destinée aux développeurs et administrateurs système chargés d'installer, de configurer et de maintenir l'application après sa livraison.

11.4 Documentation Utilisateur

Le quatrième ensemble de livrables concerne la documentation destinée aux utilisateurs finaux de l'application, c'est-à-dire l'équipe administrative de l'école, qui utilisera MédiaStock au quotidien pour la gestion des prêts et restitutions de matériel.

- **Guide d'utilisation complet** : présentation détaillée des fonctionnalités principales (création et gestion des prêts, restitutions, gestion de l'inventaire).
- **Fiches de référence rapide** : supports synthétiques au format A4, destinés à faciliter la prise en main de l'application.

11.5 Éléments Opérationnels

Le cinquième ensemble de livrables regroupe les éléments directement exploitables pour la mise en service et l'utilisation concrète de l'application MédiaStock.

- **Générateur d'étiquettes QR** : fichiers PDF imprimables permettant de produire les QR codes associés aux items du matériel.
- **Jeu d'items de démonstration** : base de données d'exemple utilisée pour les tests et la formation des utilisateurs.

11.6 Application Déployée

Le livrable final, et le plus important, est l'application MédiaStock elle-même, déployée en production et pleinement opérationnelle sur l'infrastructure de l'école.

- **Accès à l'application** : disponible via une URL sécurisée en HTTPS.
- **Base de données initialisée** : contenant les données de référence nécessaires au démarrage du service.
- **Session de formation** : organisée à destination de l'équipe administrative, afin d'assurer une prise en main efficace de l'application.

Annexes

Figure 1: MCD de la base de données

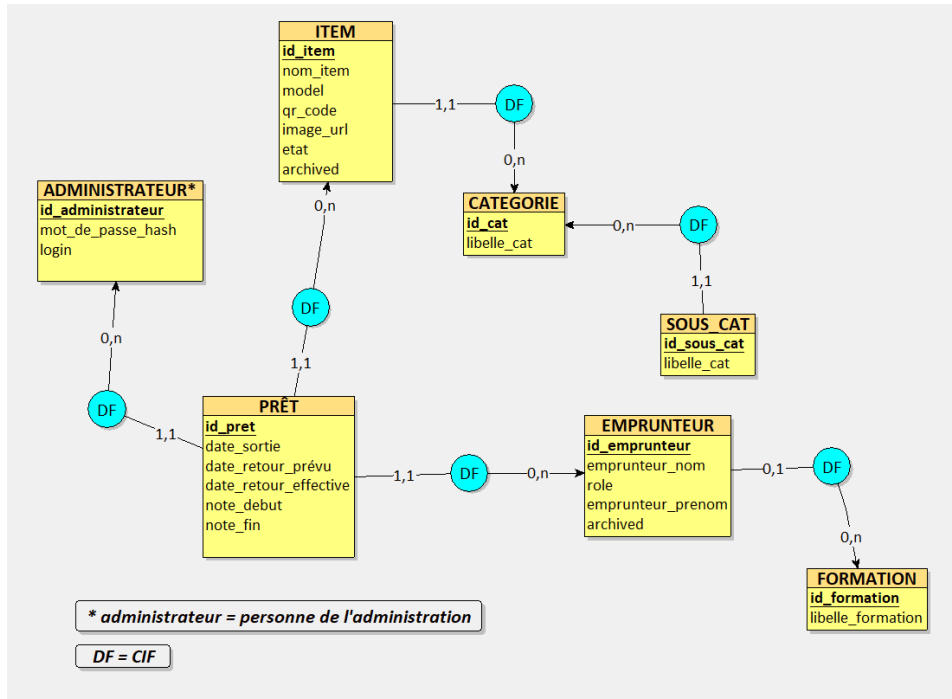


Figure 2: MLD de la base de données

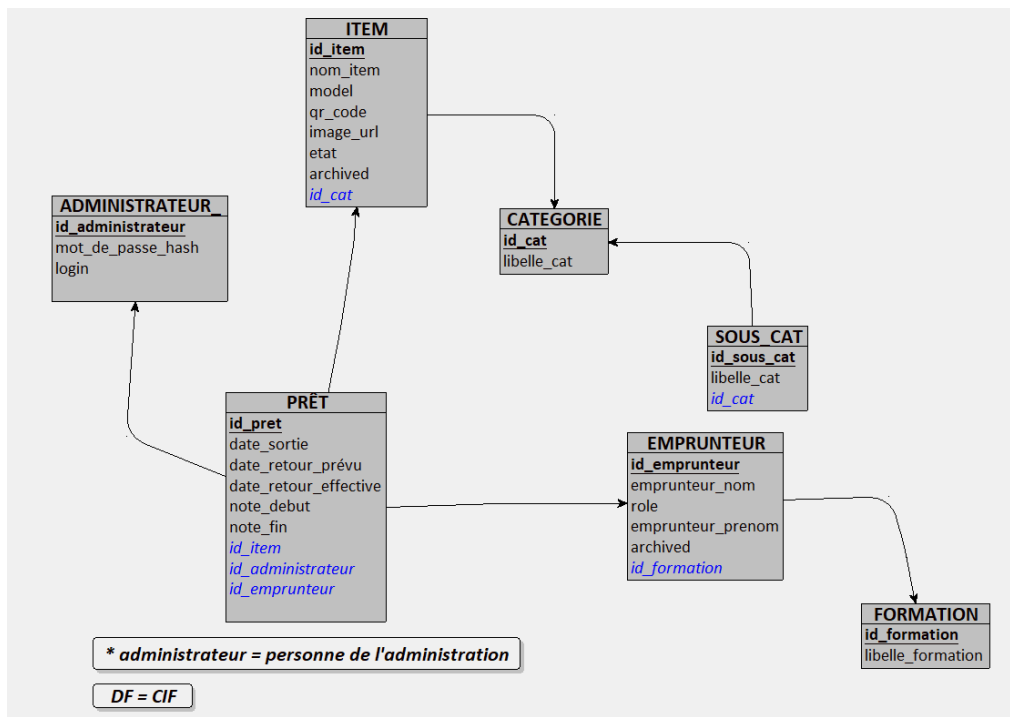
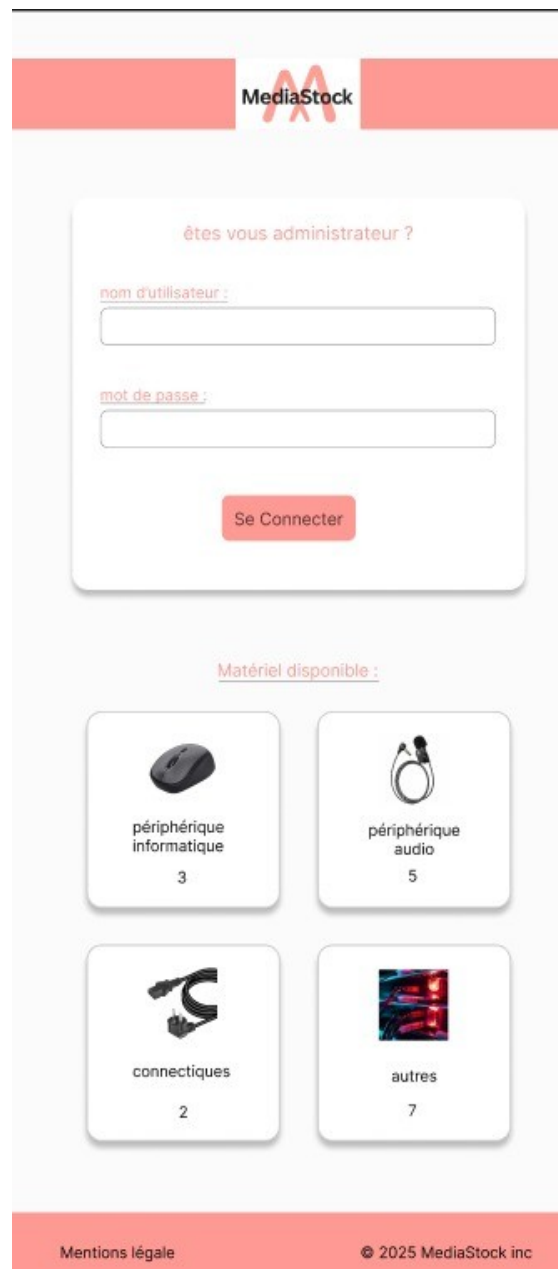


Figure 3: La maquette de l'application
MédiaStock



Maquette de l'application MédiaStock. L'interface est divisée en trois sections principales : une barre d'en-tête, une zone de connexion et une section de matériel disponible.

Barre d'en-tête : Le logo MédiaStock est centré sur une barre rouge.

Zone de connexion : Un formulaire blanc avec un titre "êtes vous administrateur ?" en rouge. Il contient deux champs de saisie : "nom d'utilisateur :" et "mot de passe :". En dessous se trouve un bouton rouge "Se Connecter".

Matériel disponible : Une section intitulée "Matériel disponible :" qui présente quatre catégories de matériel, chacune avec une icône, un titre et un nombre :

- périphérique informatique :** 3 (icône d'une souris)
- périphérique audio :** 5 (icône d'un casque)
- connectiques :** 2 (icône d'un câble)
- autres :** 7 (icône d'un boîtier électronique)

Barre de pied de page : Une barre rouge contenant le lien "Mentions légale" et le copyright "© 2025 MédiaStock inc".

Si vous souhaitez voir davantage de maquettes des pages, veuillez consulter le site ci-dessous :
<https://www.figma.com/design/8YYwxKWra3P9QWC6UJBv2L/Untitled?node-id=1-3&t=jJ7vbjfyRZe8E0c2-1>

Figure 4: Le diagramme de Gantt
MediaStock 03/10/2025 - 24/10/25

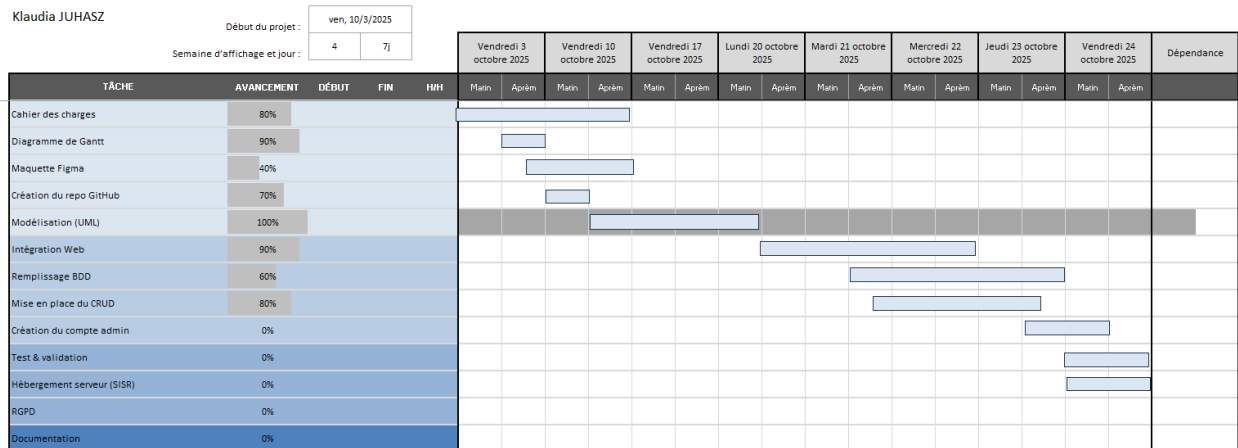


Figure 5: Diagramme de Kanban – Partie 1

Tâche	Phase	Type	Priorité	Sélectionner
Cahier des charges (finalisation)	Phase 1 : Préparation & Conception	Doc	Must	Semaine 1
Maquettes Figma (mobile-first)	Phase 1 : Préparation & Conception	Front	Must	Semaine 2
Modélisation BDD (MER)	Phase 1 : Préparation & Conception	Back	Must	Semaine 3
Repo GitHub + organisation projet	Phase 1 : Préparation & Conception	Doc	Must	Semaine 4
RGPD (politique de conservation, suppression sur demande)	Phase 1 : Préparation & Conception	Doc	Must	Semaine 5
Création BDD (MySQL) + intégration	Phase 2 : Développement & Intégration	Back	Must	Semaine 6
CRUD Inventaire (ajout, modif, suppression matériel)	Phase 2 : Développement & Intégration	Back	Must	Maison
Génération QR Code	Phase 2 : Développement & Intégration	Back	Must	
Gestion compte admin (unique partagé)	Phase 2 : Développement & Intégration	Back	Must	
Liaison BDD ↔ sécurité	Phase 2 : Développement & Intégration	Back	Must	
Hébergement serveur SISR	Phase 2 : Développement & Intégration	Back	Must	
Catalogue public (liste dispo/indispo)	Phase 2 : Développement & Intégration	Front	Must	
Connexion admin	Phase 2 : Développement & Intégration	Front	Must	
Page Inventaire (CRUD)	Phase 2 : Développement & Intégration	Front	Must	
Page Emprunter (scan QR → création prêt)	Phase 2 : Développement & Intégration	Front	Must	
Page Restituer (scan QR → retour prêt)	Phase 2 : Développement & Intégration	Front	Must	
Ajout échéance + alerte retards	Phase 2 : Développement & Intégration	Back	Should	
Impression des QR codes	Phase 2 : Développement & Intégration	Back	Should	
Filtres & recherche catalogue public	Phase 2 : Développement & Intégration	Front	Should	
Statistiques (objets les plus empruntés, durée moyenne, retards)	Phase 2 : Développement & Intégration	Back	Could	
UI charte graphique école	Phase 2 : Développement & Intégration	Front	Could	
Tests CRUD matériel	Phase 3 : Tests & Mise en Production	Test	Must	
Tests prêt (scan création)	Phase 3 : Tests & Mise en Production	Test	Must	
Tests restitution (scan retour)	Phase 3 : Tests & Mise en Production	Test	Must	

Figure 6: Diagramme de Kanban – Partie 2

Tests CRUD matériel	Phase 3 : Tests & Mise en Production	Test	Must
Tests prêt (scan création)	Phase 3 : Tests & Mise en Production	Test	Must
Tests restitution (scan retour)	Phase 3 : Tests & Mise en Production	Test	Must
Test RGPD (anonymisation/suppression)	Phase 3 : Tests & Mise en Production	Test	Must
Validation mobile-first sur plusieurs appareils	Phase 3 : Tests & Mise en Production	Test	Must
Documentation utilisateur (guide prêt/restitution)	Phase 4 : Finalisation	Doc	Should
Organisation tâches (back / front / tests / RGPD)	Phase 4 : Finalisation	Doc	Must
Contexte projet défini (objectif traçabilité prêts/restits)	Phase 4 : Finalisation	Doc	Must
Choix techno validé : MySQL + PHP + responsive web	Phase 4 : Finalisation	Doc	Must
Planning jalons (avant Noël 2025)	Phase 4 : Finalisation	Doc	Must
+ Nouvelle page			